Manual del estudiante — Semana 1

El hilo: el modelo, la configuración y la línea base

Proyecto final de Razonamiento Automático · Prof. Fabián Serrano
Asesoría: Carlos Orrego · UNAL Manizales · 5 de noviembre de 2026

La idea en una frase

Al terminar esta semana cada uno tiene un modelo de lenguaje gratuito respondiendo desde Python y sabe medir qué tan mal demuestra cuando nadie lo ayuda. Ese número se llama **línea base**, y todo el proyecto se mide contra él.

Cómo usar este manual. Síganlo en orden, con la terminal abierta. Cada caja azul es algo que hay que hacer; cada caja naranja, un error típico y cómo salir de él. Todo el código está en el repositorio: `https://github.com/cmorregof/repo_proyecto_final`.

1. La idea del proyecto

Un modelo de lenguaje predice la siguiente palabra más probable. Por eso escribe con fluidez, pero lo que escribe es *plausible*, no necesariamente *verdadero*. Cuando afirma algo falso con total seguridad, se dice que **alucina**. Isabelle hace lo contrario: no se deja convencer por el estilo, acepta o rechaza, y dice dónde falla.

El proyecto junta a los dos: **el modelo propone; Isabelle decide.** El asistente que van a construir busca en los documentos del curso (eso es un *RAG*: un examen a libro abierto), le pide al modelo una demostración, y se la hace verificar a Isabelle antes de mostrarla. Si Isabelle la rechaza, el error vuelve al modelo para que corrija.

Jueves	Nombre	Qué se construye
1	**El hilo**	**el modelo y la línea base (configuración 1)**
2	La biblioteca	buscar en los documentos y citar (configuración 2)
3	La flecha roja	verificar con Isabelle y corregir (configuración 3)
4	El espejo	evaluar, analizar errores, informe

Se califica **la mejora** de la configuración 1 a la 3 con el mismo modelo, no el porcentaje absoluto: un modelo gratuito pequeño compite en igualdad con uno de pago. Y por encima de todo, la honestidad: el asistente nunca dice «verificado» sin que Isabelle lo haya aceptado.

2. Preparar el computador

Lo que necesitan

1. **Isabelle** (el del curso).
2. **Python 3.9 o posterior.** No hay que instalar paquetes.
3. **git** (o descargar el repositorio como ZIP).
4. **Un modelo de lenguaje gratuito**: Ollama (recomendado) u OpenRouter.

Descargar el repositorio

Hágalo

```
git clone https://github.com/cmorregof/repo_proyecto_final.git
cd repo_proyecto_final
```

Sin git: botón verde **Code** → **Download ZIP** en la página del repositorio. Si ya lo habían clonado, actualicen con `git pull`.

Decirle a Python dónde está Isabelle

Hágalo

Si escribir `isabelle version` en la terminal no funciona, indiquen la ruta (ajústenla a su instalación y versión):

```
# Mac
export ISABELLE=/Applications/Isabelle2025-2.app/bin/isabelle
# Linux
export ISABELLE=$HOME/Isabelle2025-2/bin/isabelle
```

Hay que repetirlo en cada terminal nueva (o agregarlo a `~/.zshrc` o `~/.bashrc`).

Camino A: Ollama con Gemma 3 (recomendado)

Ollama es un programa gratuito que descarga modelos y los corre en su propio computador: sin límite de uso, sin llave, sin costo y sin internet. Gemma 3 4B funciona en un portátil de 8 GB de RAM, sin tarjeta gráfica (lento, pero funciona).

Hágalo

1. Instalar Ollama desde `https://ollama.com` (Mac: también `brew install ollama`).
2. Descargar el modelo, una sola vez (unos 3 GB):

   ```
   ollama pull gemma3:4b
   ```

3. Dejar el servidor corriendo **en una terminal aparte** que no se cierra mientras trabajan:

   ```
   ollama serve
   ```

Si sale: `address already in use`

Ollama ya está corriendo (por ejemplo, la aplicación). No hay que hacer nada.

Camino B: OpenRouter

Para quien no tenga un computador que aguante el modelo local. Creen una cuenta en `https://openrouter.ai`, generen una llave y expórtenla:

```
export OPENROUTER_API_KEY=sk-or-...
```

Luego, en `semana1/taller/modelo.py`, cambien `"proveedor"` a `"openrouter"` y `"modelo"` a un modelo cuyo nombre termine en `:free`.

Ojo

La llave es una contraseña: **nunca** la escriban dentro del código ni la suban a GitHub. Las cuentas gratuitas tienen un límite diario (del orden de 50 solicitudes); una corrida completa del proyecto puede pedir 400. Por eso el camino A es el principal.

Otros modelos

Cualquier modelo pequeño y abierto sirve: `qwen2.5-coder:7b` (sabe de código, pide unos 16 GB), `qwen3:4b`, `phi4-mini`, `llama3.2:3b` (para computadores modestos). Se cambia el nombre en `CONFIG`. **Regla:** las tres configuraciones del informe, con el mismo modelo.

3. El kit del profesor: la carpeta `Recursos/`

`verificador.py`	le pregunta a Isabelle si un texto es correcto
`benchmark_R1.json`	las 58 preguntas del benchmark público
`evaluar.py`	califica sus respuestas (el mismo que usará el profesor)
`ejemplo_respuestas.json`	el formato de respuestas
`README.md`	instrucciones detalladas: léanlo

Probar el verificador

Hágalo

Desde la carpeta `Recursos/`:

```
from verificador import verificar

r = verificar('lemma "p \\<and> q \\<Longrightarrow> q \\<and> p" by auto')
print(r.ok)          # True si Isabelle la acepta
print(r.segundos)    # unos 4 o 5
print(r.errores)     # lista vacia si r.ok
```

El verificador acepta una teoría completa o solo un pedazo, convierte los símbolos Unicode (∧, ⟶) a la forma que exige Isabelle (`\<and>`…), da los errores con la línea del texto que ustedes enviaron, corta las pruebas que no terminan y no deja pasar `sorry`.

Cómo es una pregunta y qué se responde

Cada pregunta del benchmark tiene `id`, `estilo` (`"dn"`, deducción natural, o `"libre"`), `enunciado` y `contexto` (todo lo que va antes del enunciado en la teoría: cabecera y lemas previos).

La respuesta es **solo lo que va después del enunciado**, desde `proof`, `by`, `using` o `apply`. El evaluador arma `contexto + enunciado + respuesta + end`.

`aceptada`	Isabelle la acepta y respeta el estilo
`estilo`	la acepta, pero usa `auto` donde se pedía deducción natural
`rechazada`	Isabelle no la acepta; el informe trae el error
`trampa`	contiene lemas, axiomas, `sorry` u otros comandos: ni se verifica
`sin respuesta`	falta

4. El taller: la configuración 1

Todo está en `semana1/taller/`. Son cuatro archivos y dos programas:

`modelo.py`	`CONFIG` y `preguntar()`: hablar con el modelo
`utilidades.py`	`extraer_prueba()` y `registrar()`
`prompt.py`	`pedir_mensajes()`: armar la pregunta
`config1.py`	corre el benchmark con solo el modelo
`paso1c.py`	primera conversación de prueba

Paso 1: hablar con el modelo

En `modelo.py`, todo lo que cambia entre un modelo y otro vive en un diccionario:

```
CONFIG = {
    "proveedor":   "ollama",       # o "openrouter"
    "modelo":      "gemma3:4b",    # o el nombre de otro modelo
    "temperatura": 0.2,            # baja = respuestas mas estables
}
```

La función `preguntar(mensajes)` recibe una lista de mensajes y devuelve el texto de la respuesta. Ollama y OpenRouter usan el mismo formato, así que el resto del código no cambia. Los mensajes tienen **rol**: `system` (quién es el modelo y sus reglas), `user` (la pregunta) y `assistant` (lo que el modelo ya contestó; lo usaremos la semana 3).

Hágalo

```
cd semana1/taller
python3 paso1c.py
```

Debe aparecer una explicación de la regla `mp`.

Si sale: `Connection refused`

Ollama no está corriendo. Abran otra terminal, escriban `ollama serve` y déjenla abierta.

Si sale: `No such file or directory`

No están en la carpeta correcta. `cd` hasta `repo_proyecto_final/semana1/taller` y vuelvan a intentar.

Lean la respuesta con lupa

En el ensayo, Gemma 3 4B respondió a «¿qué dice la regla `mp`?» con una explicación correcta de modus ponens… y luego un «ejemplo en Isabelle/Isar» con `module Main`, `assert`, `check` e `infer [mp]`, y unas reglas `and` y `or`. **Nada de eso existe en Isabelle**: una teoría empieza con `theory ... imports Main begin`, `mp` se usa con `using 1 2 by (rule mp)`, y las reglas de la conjunción y la disyunción son `conjI`, `conjE`, `disjI1`, `disjE`. Todo dicho con la misma seguridad.

Ojo

Eso es una alucinación: la mitad es verdad y la otra mitad es «Isabelle imaginario», sin que el tono cambie. Por eso este proyecto existe.

Paso 2: limpiar la respuesta y dejar registro

Los modelos rodean la prueba de explicaciones y comillas triples, y **a veces repiten el enunciado aunque se les pida que no**. `extraer_prueba` saca el bloque de código; si empieza con `lemma`, corta hasta la primera línea que empiece con `proof`, `by`, `using` o `apply`; y quita un `oops` final.

`registrar` agrega una línea JSON por evento al archivo de registro (modo `"a"`: se agrega, nunca se borra). **Sin registros no se califica** (requisito R5).

Paso 3: armar la pregunta

`pedir_mensajes(p, extra="")` junta el contexto, un espacio `extra`, el enunciado y la regla de estilo (si es `dn`: solo `rule`, `erule`, `assumption`...; nada de `auto`, `simp`, `blast`). Hoy `extra` va vacío; la semana 2 ahí entran los pasajes que encuentre la búsqueda.

Paso 4: correr y calificar

Hágalo

Primero con 3 preguntas, para comprobar que todo funciona:

```
python3 config1.py 3
python3 ../../Recursos/evaluar.py ../../Recursos/benchmark_R1.json \
        respuestas_config1.json --paralelo 4 --informe informe_c1.json
```

Cuando funcione, las 58: `python3 config1.py` y de nuevo `evaluar.py`.

Ojo

Con Gemma 3 4B en un portátil sin tarjeta gráfica, las 58 tardaron unos **22 minutos** en el ensayo: la mayoría tarda segundos, pero algunas respuestas se van en bucle y tardan hasta 3 minutos. Déjenlo corriendo y hagan otra cosa.

Lo que pasó en el ensayo: 44 «trampas»

Con una primera versión de `extraer_prueba` que no quitaba el enunciado repetido, las 58 salieron: **0 aceptadas, 0 de estilo, 14 rechazadas y 44** `trampa`. Gemma no hacía trampa a propósito: desobedecía la instrucción «sin repetir el enunciado» y copiaba el `lemma`, así que el evaluador ni la mandaba a Isabelle.

La lección: una instrucción en el prompt no es una garantía. Lo que el modelo *debe* cumplir lo verifica el programa. Limpiar el formato no es hacer trampa (no se cambia la prueba, solo se quita lo que el evaluador ya pone), pero hay que declararlo en el informe.

Errores reales de Gemma que Isabelle atrapó en el ensayo:

- **Ejercicio 11:** inventa la regla `implicacion_con_consecuencia`; Isabelle: `Undefined fact`.
- **Ejercicio 12:** inventa la regla `imply`.
- **Ejercicio 1:** `using 1, 2 by rule` falla solo por la coma; sin ella, Isabelle la acepta.
- **Ejercicio 1, otra variante:** `using 2 1 by (rule mp)` falla con `Failed to apply initial proof method`, porque `mp` espera primero la implicación y luego la premisa.

El número que obtengan con la limpieza corregida es su línea base. Anótenlo.

5. Temas de formalización

Cada grupo (máximo tres) toma un tema distinto. Los ejercicios de cada grupo serán el examen de los asistentes de los demás: al menos 8, dos de ellos `[***]`.

Rama	Teorema central	Nivel
Codificación	`decodificar (codificar xs) = xs` (longitud de racha)	⋆
Criptografía clásica	César o Vigenère: descifrar invierte a cifrar	⋆
Algoritmos	Ordenamiento por inserción: ordena y es permutación	⋆⋆
Estructuras de datos	Árboles de búsqueda: la inserción preserva el orden	⋆⋆
Compiladores	Expresiones a máquina de pila: `exec (compilar e) [] = [eval e]`	⋆⋆
Lenguajes formales	Palíndromos o paréntesis balanceados	⋆⋆
Lógica computacional	La forma normal negativa preserva el significado	⋆⋆
Arquitectura	Sumador binario sobre listas de bits	⋆⋆⋆
Grafos	Alcanzabilidad como cierre reflexivo-transitivo	⋆⋆⋆
Lenguajes de programación	Tipado de expresiones: preservación	⋆⋆⋆
Teoría de la computación	Autómatas: el producto reconoce la intersección	⋆⋆⋆
Software (entusiastas)	Verificar un programa imperativo (ver *Concrete Semantics*)	⋆⋆⋆

6. Tarea para el próximo jueves

Hágalo

1. Configuración 1 completa sobre las 58: `respuestas_config1.json`, `registros/config1.jsonl` e `informe_c1.json`.
2. Leer diez registros a mano y clasificar los errores del modelo: sintaxis, reglas inventadas, orden de los hechos, `auto` prohibido, enunciado repetido… Es material del informe.
3. Tema de formalización: tres opciones en orden de preferencia, y los dos primeros enunciados escritos en Isabelle.

La verdad no la pone el modelo más caro: la pone Isabelle.